

虚函数与消息的动态绑定

1.消息的多态性

- 消息的多态性体现为：
 - 相同的一条消息可以发送到不同类的对象，从而会得到不同的解释（处理）
- 对于具有 *public* 继承关系的两个类：
 - 一条可以发送到基类对象的消息，也可以发送到派生类对象
 - 如果在基类和派生类中都给出了对这条消息的处理，那么这条消息存在特殊的多态性

2.消息的静态绑定

一般情况下，将在编译时刻根据对象的类型来决定采用哪一个消息处理函数，即采用静态绑定

```
1  class A
2  {   public:
3      void f();
4  };
5
6  class B: public A
7  {   public:
8      void f();
9      void g();
10 };
11 .....
12 A a;
13 B b;
14 a.f(); //A的f
15 b.f(); //B的f
16 b.A::f(); //A的f
```

3.消息的动态绑定

由于基类的指针或引用可以指向或引用基类对象，也可以指向或引用派生类对象。

通过基类的指针或引用向它指向或引用的对象发送消息时，调用哪一个消息处理函数：

基类的？ 派生类的？

```
1  class A
2  {   int x,y;
3      public:
4          void f();
5  };
6
7  class B: public A
8  {   int z;
9      public:
10     void f();
11     void g();
12 };
13
14 void func1(A& x)
```

```

15 { .....
16     x.f(); //调用A::f还是B::f ? 答案是: A::f
17     .....
18 }
19 void func2(A *p)
20 { .....
21     p->f(); //调用A::f还是B::f ? 答案是: A::f
22     .....
23 }
24 .....
25 A a;
26 func1(a);
27 func2(&a);
28 B b;
29 func1(b);
30 func2(&b);
31
32 C++默认的是静态绑定!

```

一般情况下，需要在运行时刻，根据func1（或func2）中x（或p）实际引用（或指向）的对象来决定是调用A::f还是B::f，即采用动态绑定。

- 在C++中，在基类中用虚函数来指出动态绑定

```

1 class A
2 {     int x,y;
3     public:
4         virtual void f(); //虚函数
5 };
6
7 class B: public A
8 {     int z;
9     public:
10        void f();
11        void g();
12 };
13
14 void func1(A& x)
15 {     .....
16     x.f(); //调用A::f或B::f
17     .....
18 }
19 void func2(A *p)
20 {     .....
21     p->f(); //调用A::f或B::f
22     .....
23 }
24 .....
25 A a;
26 func1(a); //在func1中调用A::f
27 func2(&a); //在func2中调用A::f
28 B b;
29 func1(b); //在func1中调用B::f
30 func2(&b); //在func2中调用B::f
31

```

4.虚函数

虚函数是指加了关键词 `virtual` 的成员函数

格式：`virtual <成员函数声明>;`

作用：

- 指定消息采用动态绑定。
- 指出基类中可以被派生类重定义的成员函数。

4.1 在派生类中重定义基类的成员函数

对于基类中的一个虚函数，在派生类中定义的、与之具有相同型构的成员函数是对基类该成员函数的重定义（或称覆盖，`override`）

相同的型构是指：

- 派生类中定义的成员函数的名字、参数个数和类型与基类相应成员函数相同；
- 其返回值类型与基类成员函数返回值类型或者相同，或者是基类成员函数返回值类型的`public`派生类。

消息的动态绑定将绑定到派生类中与基类同型构的成员函数！

```
1  class A
2  {   int x,y;
3      public:
4          virtual void f();
5  };
6
7  class B: public A
8  {   int z;
9      public:
10         void f(); //对A中f的重定义
11         void f(int); //新定义的成员函数
12         void g();
13  };
14
```

4.2 对虚函数的几点说明

- 只有类的成员函数才可以为虚函数，但静态成员函数不能是虚函数
- 构造函数不能是虚函数，析构函数可以（往往）是虚函数
- 只要在基类中说明了虚函数，在派生类、派生类的派生类、...中，同型构的成员函数都是虚函数（`virtual`）可以不写
- 只有通过**基类的指针或引用**访问**基类的虚函数**时才进行动态绑定
- 类的构造函数和析构函数中对虚函数的调用不进行动态绑定

```
1  class A
2  {   .....
3      public:
4          A() { f(); }
5          ~A() { f(); }
```

```

6     virtual void f();
7     void g();
8     void h() { f(); g(); }
9 };
10
11 class B: public A
12 { .....
13 public:
14     B();
15     ~B();
16     void f();
17     void g();
18 };
19
20 .....
21 A a; //调用A::A()和A::f
22 a.f(); //调用A::f
23 a.g(); //调用A::g
24 a.h(); //调用A::h、A::f和A::g
25 //a消亡时会调用A::~~A()和A::f
26
27 B b; //调用B::B()、A::A()和A::f
28 b.f(); //调用B::f
29 b.g(); //调用B::g
30 b.h(); //调用A::h、B::f和A::g
31 //b消亡时会调用B::~~B()、A::~~A()和A::f
32
33 A *p; //p是A类（基类）指针
34 p = &a; //p指向A类对象
35 p->f(); //调用A::f
36 p->g(); //调用A::g
37 p->h(); //调用A::h、A::f和A::g
38
39 p = &b; //p指向B类对象
40 p->f(); //调用B::f
41 p->A::f(); //调用A::f，类名受限采用静态绑定
42 p->g(); //调用A::g，非虚函数采用静态绑定
43 p->h(); //调用A::h、B::f和A::g
44 p = new B; //调用B::B()、A::A()和A::f
45 .....
46 delete p; //只调用A::~~A()和A::f，
47           //没调用B::~~B()，为什么？
48           //没有把A的析构函数定义为虚函数！
49

```

- 通过基类指针访问派生类中新定义的成员

```

1 class A
2 { .....
3     public:
4         virtual void f();
5 };
6 class B: public A
7 { .....
8     public:

```

```

9      void f();
10     void g();
11 };
12 A *p=new B;
13 .....
14 p->f(); //OK
15 p->g(); //Error
16 ((B *)p)->g(); //OK, 不安全! 万一p指向的不是B类对象呢?
17
18 安全做法:
19 A *p;
20 .....
21 B *q=dynamic_cast<B *>(p);
22 if (q != NULL) q->g();
23 如果p指向的不是B类对象, q为空
24 该操作需要运行时刻的类型信息支持
25 (有编译选项: RTTI)

```

4.3何时需要定义虚函数

- 基类中哪些成员函数需要设计成虚函数？
 - 在设计基类时，有时虽然给出了某些成员函数的实现，但实现的方法可能不是最好，今后可能还会有更好的实现方法。
 - 在基类中根本无法给出某些成员函数的实现，它们必须由不同的派生类根据实际情况给出具体的实现。（抽象类与纯虚函数）